Failure Localization for Enterprise Text-to-SQL Agents: Lessons from a Production-Representative Financial Case Study

Heeyong Eun[1]

KPMG Korea, Seoul, South Korea
heeyonge@gmail.com

Abstract. Large-language-model Text-to-SQL agents are attractive AI-assisted software engineering tools, yet enterprise deployments often fail *silently*: generated SQL can execute while answering the wrong intent due to mis-grounded terms, wrong anchors, meaningless joins, or ungrounded filters. The core challenge is diagnostic because failures can originate at different pipeline stages, but typical retrieval-and-generation systems do not preserve intermediate commitments in an inspectable form. I present an artifact-level failure-localization method that fixes major intermediate decisions as structured artifacts and compares them with reference artifacts derived from administrator-corrected SQL via a deterministic "Gold Reverser." Paired replay of 69 queries on a production-representative financial metadata snapshot shows improved schema linking and fewer structural errors than a baseline. During a subsequent one-month practice operation with unseen requests, the artifacts exposed the failing stage and concrete commitment, while also revealing inefficient SQL and omitted implicit predicates. These observations motivate incrementally converting DA/DBA-reviewed SQL into reference and regression artifacts.

Keywords: Text-to-SQL; enterprise LLM agents; AI for software engineering; failure localization; regression testing; schema linking; structural validation

1 Introduction

Text-to-SQL agents translate natural-language requests into executable database queries and are increasingly deployed as AI-assisted software engineering tools that produce code-like artifacts. In enterprise settings, however, a generated SQL query can be *executable* yet still be *wrong*: it may answer a different business question, ground a term to an incorrect physical column, choose an inappropriate anchor table, construct a plausible-but-meaningless join, or introduce an unsupported filter. This gap is amplified in large, heterogeneous schema environments compared with benchmark-style scenarios such as Spider [5] and schema-linking approaches such as RAT-SQL [2].

The core operational challenge is diagnostic. Failures can originate at multiple stages—term interpretation, schema grounding, table selection and join planning,

or filter construction—but typical pipelines do not preserve intermediate commitments in an inspectable form. Decisions are absorbed into prompts or model traces and only surface in the final SQL, making it difficult to localise where the pipeline first diverged. Teams then resort to repeated prompt or rule patching, which is hard to reproduce and improve cumulatively.

I make one claim. If intermediate decisions are fixed as structured artifacts and compared against reference artifacts derived from administrator-corrected SQL, failures can be localised to the *first wrong commitment* (e.g., grounding vs. linking vs. join vs. filter), exposing the specific incorrect or missing column, table, join edge, or predicate for targeted correction. This claim aligns with the debugging intuition that isolating failure-inducing differences enables systematic correction [6]. I study this claim through paired replay of 69 queries and a subsequent one-month practice operation with unseen requests (Section 3). The operation exposed inefficient SQL and omitted implicit predicates beyond schema-level failures, motivating continuous extension of the reference corpus with DA/DBA-reviewed SQL.

2 Background and Related Work

Enterprise-scale schema linking shifts the bottleneck upstream. Benchmark progress shows that strong models can produce complex SQL when the target schema is bounded (e.g., Spider) [5]. Enterprise deployments change the regime: the difficulty often becomes filtering large schema pools and grounding terms to the correct subset of tables and columns under redundancy. LinkAlign frames scalable schema linking as database retrieval/schema pool filtering plus schema item grounding in large multi-database environments [4].

Multi-step pipelines amplify intermediate errors without control surfaces. When Text-to-SQL is embedded in multi-stage agent pipelines, incorrect intermediate outputs can propagate forward and become unchallengeable assumptions downstream. GUARDIAN highlights error injection and amplification in multi-agent collaboration, motivating engineered control surfaces for verification and correction [7].

Why artifact-level localisation is an AI4SE mechanism. Delta debugging formalises localising failure-inducing differences via systematic comparison [6]. I adopt the same diagnostic stance for Text-to-SQL by comparing intermediate commitments (grounded columns, selected tables, join paths, filters) against administrator-corrected references. This complements emerging views that LLM applications should be treated as systems requiring explicit testing protocols rather than only aggregate accuracy [1].

3 Industrial Case and Problem Definition

Setting and boundaries. I study an enterprise Text-to-SQL workload from a financial organisation under strict operational constraints. Initial development and

paired evaluation use a production-representative *metadata snapshot* (900 tables, 22,000 columns, 24,000 domain terms, 3,000-word lexicon) and 69 anonymised business queries with administrator-approved SQL. Customer records and transaction data are excluded; the evaluation concerns schema/terminology grounding and SQL structure. This boundary is deliberate: enterprise Text-to-SQL difficulty is often dominated by schema pool filtering and grounding under redundancy and ambiguity [4].

What "production-representative" means here. Schemas and terminology reflect operational naming conventions and overlaps, and reference SQL is derived from administrator-approved or routinely executed queries. After paired replay, the revised agent entered a one-month practice operation with unseen requests beyond the 69-query suite. Its stage-wise artifacts supported investigation and correction; the original Gold SQL was not supplied during inference.

Baseline vs. revised agent (high-level). The *baseline* retrieves schema evidence and generates SQL end-to-end. The *revised* configuration records grounding, table selection, join planning, and filtering as inspectable artifacts. Gold-derived references support post-generation diagnosis and paired evaluation but are not supplied during inference.

Implementation configuration. The guided workflow uses `gpt-4.1-mini` for query interpretation and `gpt-4o` for join verification, planning, and synthesis (temperature 0), without Text-to-SQL fine-tuning. Stage-specific prompts select only catalog-grounded candidates from frozen metadata and emit schema-fixed JSON artifacts consumed by downstream stages.

Problem definition. Enterprise failures frequently remain *silent*: SQL executes but answers the wrong intent due to mis-grounded terms, incorrect anchoring, meaningless joins, or ungrounded filters. The objective is to make these failures testable as software: localise the first wrong commitment and enable targeted correction. During practice operation, artifacts exposed the failing stage and concrete commitment. They also revealed defects beyond schema linking: inefficient but result-equivalent SQL and omitted business predicates such as `DEL_YN = 'N'`.

4 Method: Artifact-Level Failure Localization

The comparison identifies the pipeline stage and concrete artifact to inspect; it neither assumes a reference before generation nor automatically retrains the model.

4.1 Pipeline commitments as artifacts

Artifact schema. I fix intermediate commitments as JSON artifacts emitted after major stages: (i) grounding artifact $\mathcal{A}_g$ (committed physical columns and

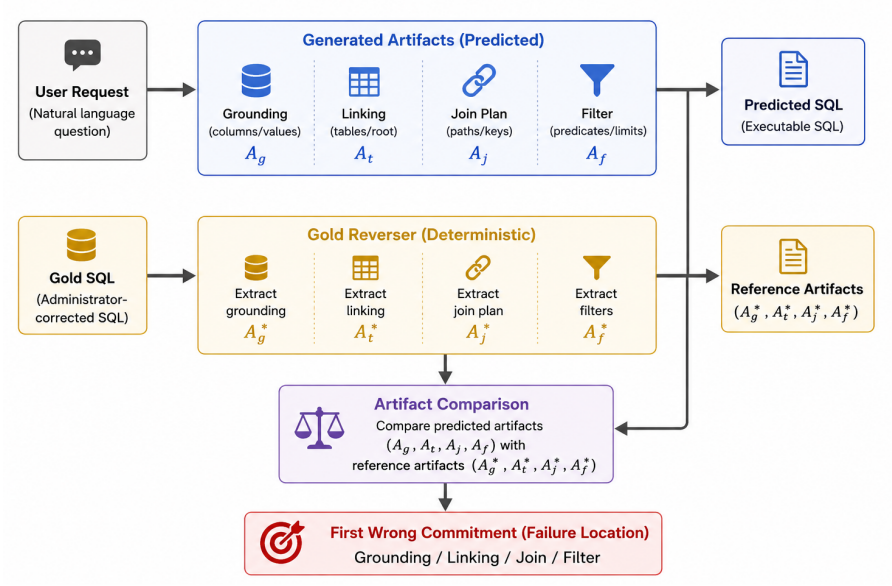

Fig. 1. Artifact-level failure localization: compare generated artifacts with gold-derived reference artifacts to label the first wrong commitment (grounding/linking/join/filter).

optional value bindings), (ii) linking artifact $\mathcal{A}_t$ (selected tables, root/anchor, column-to-table bindings), (iii) join-plan artifact $\mathcal{A}_j$ (join edges and keys under admissibility rules), and (iv) filter artifact $\mathcal{A}_f$ (WHERE predicates and optional ORDER/LIMIT decisions). Downstream stages consume artifacts rather than free-form traces: $\mathcal{A}_g \rightarrow \mathcal{A}_t \rightarrow \mathcal{A}_j \rightarrow \mathcal{A}_f \rightarrow SQL$.

4.2 Gold SQL to reference artifacts (Gold Reverser)

Gold SQL and Gold Reverser. Gold SQL denotes administrator-approved or production-derived SQL under the same snapshot. The deterministic *Gold Reverser* extracts `FROM`/`JOIN` tables, `SELECT`-list columns, the `WHERE` snippet, and `LIMIT` presence using lightweight rules. These commitments populate the grounding, linking, and filter references, while the frozen join graph supplies admissible join constraints. Separate evaluation scripts use `sqlglot` to extract identifiers and validate parsing, catalog membership, and join structure. Complex aliases, subqueries, vendor-specific constructs, views, and parameters require adaptation or manual review; reverser output is therefore reviewable rather than an infallible oracle.

Why artifact matching instead of SQL-string matching. Structurally different SQL may be semantically equivalent, and enterprise SQL may require alternative bridge tables; strict string or set matching can therefore penalise valid queries. I therefore

compare artifacts while separately reporting structural validity, consistent with enterprise linking constraints [4].

4.3 Divergence rules: localising the first wrong commitment

Deterministic divergence classification. Given predicted artifacts $(\mathcal{A}_g, \mathcal{A}_t, \mathcal{A}_j, \mathcal{A}_f)$ and references $(\mathcal{A}_g^\star, \mathcal{A}_t^\star, \mathcal{A}_j^\star, \mathcal{A}_f^\star)$, I label the first wrong commitment in this order: (1) grounding failure (missing required columns, non-existent columns, or value bindings not grounded), (2) linking failure (wrong root/anchor or invalid bindings), (3) join violation (disconnected join islands or *inadmissible* join edges/keys), (4) filter invention/violation (unsupported predicates or ORDER/LIMIT without evidence). Here, *inadmissible* join edges/keys are those not permitted by the frozen join graph (or declared join-key constraints) used for the snapshot. The ordering prevents blaming downstream stages when upstream commitments already diverged.

Use during practice operation. For a previously unseen request, no Gold SQL is assumed at generation time. If the generated SQL is rejected or requires investigation, the stored artifacts expose the committed columns, tables, joins, and filters, allowing the reviewer to identify the first problematic stage and the concrete incorrect or missing item. A DA/DBA-approved SQL can then be reversed into reviewable reference commitments and added to the regression corpus. This paper proposes this human-governed accumulation step as an operational correction workflow; it is distinct from automatic prompt updating or model retraining.

5 Running Examples

I provide two anonymised examples that make artifact divergence visible. Each example shows the user request, a compact SQL fragment, and the artifact-level divergence label.

5.1 Example A: Semantic grounding failure (term drift)

User request (anonymised). [REQ-A] "List the current status of `[ENTITY]` accounts opened in `[PERIOD]`, excluding closed ones."

Administrator-corrected SQL (fragment).

```
SELECT ..., a.current_status_code
FROM ...
WHERE a.current_status_code <> 'CLOSED' ...
```

Baseline divergence. The baseline grounds "status" to a historical/derived field (`status_history_code`), yielding an executable but semantically drifted query. The first wrong commitment is therefore a grounding failure.

Table 1. Example A: first divergence in the grounding artifact ($\mathcal{A}_g$).

Item	Predicted ($\mathcal{A}_g$)	Reference ($\mathcal{A}_g^\star$)
Committed column for "status"	`status_history_code`	`current_status_code`
Evidence/source (example)	lexical partial match on `status`	admin-corrected intent alignment
Impact	semantic drift with valid execution	intent-preserving grounding

Table 2. Example B: divergence in root selection ($\mathcal{A}_t$) and join admissibility ($\mathcal{A}_j$).

Signal	Predicted artifacts	Reference artifacts
Root/anchor ($\mathcal{A}_t$)	`Product` as root; bindings skew to product attributes	`Customer` as root; bindings cover customer/account identifiers
Join edges ($\mathcal{A}_j$)	includes `Customer.name = Product.owner_name` (name-based)	uses key-based joins (e.g., `customer_id`, `account_id`)
Violation label	meaningless join / weak key evidence	admissible join path

5.2 Example B: Structural failure (wrong anchor / meaningless join)

User request (anonymised). [REQ-B] "For `[ENTITY]` customers, show `[MEASURE]` by `[CATEGORY]` for `[PERIOD]`."

Administrator-corrected SQL (fragment).

```
SELECT ...
FROM Customer c
JOIN Account a ON a.customer_id = c.customer_id
JOIN Fact f ON f.account_id = a.account_id ...
```

Baseline divergence. The baseline selects an inappropriate anchor table and constructs a plausible but business-invalid join using name similarity (e.g., `customer_name`=`owner_name`). The first wrong commitment is localised at linking/join planning.

6 Evaluation

6.1 Dataset and protocol

I quantitatively evaluate the baseline and revised configurations via paired replay on the same frozen snapshot and 69 anonymised queries with administrator-approved SQL. The reference SQL uses a median of one table (mean 1.68, range 1–4) and four distinct physical columns (mean 5.28, range 2–20) per query;

Table 3. Paired replay results (N=69). Baseline vs. revised.

Metric	Baseline	Revised
CLA (Macro-F1) ↑	0.543	0.782
TLA (Macro-F1) ↑	0.327	0.547
SER ↓	0.725	0.536

33 of 69 cases (47.8%) are multi-table queries. Each configuration is executed under identical evidence resources, and reference artifacts are withheld from generation and used only for post-generation comparison. The separate one-month practice operation is reported as operational experience rather than included in the CLA/TLA/SER estimates because it accepted previously unseen requests without a pre-existing Gold SQL for every request. The protocol therefore targets diagnosability and operational correctability rather than universal SOTA claims.

6.2 Metrics

I report three compact metrics. **CLA** measures Macro-F1 over exact matching of physical column names. **TLA** measures Macro-F1 over exact matching of physical table names; in enterprise SQL, bridge tables may be necessary, so strict set-based TLA is conservative [4]. **SER** measures the rate of structural failures (parsing failure, catalog violation, or join-structure violation); execution-aware robustness motivates structural checks [3]. Because transaction rows were excluded, denotation-level end-to-end accuracy and query-plan performance were not measurable; the metrics assess schema commitments and structural safety.

6.3 Results and interpretation

Table 3 summarises baseline vs. revised (N=69) under paired replay on the frozen snapshot. Under the ordered comparison of the measured artifact levels, the first detected divergence occurred at grounding in 41 cases and linking in 15 cases. No case first diverged only at the downstream structural/join check, while 13 cases showed no divergence in the reported CLA/TLA/SER artifacts. Filter artifacts supported case-level diagnosis, but a validated aggregate filter-first count is unavailable and therefore excluded from the stage distribution.

Higher CLA/TLA indicates more correct schema commitments, and lower SER indicates fewer structurally unsafe outputs. The remaining revised error rates, particularly TLA of 0.547 and SER of 0.536, show that the system is not ready for unrestricted autonomous use. The operational value of the artifacts is therefore not that they guarantee a correct final SQL, but that they make the first divergent stage and the associated incorrect commitment inspectable, supporting targeted investigation and cumulative correction [6, 1].

7 Discussion and Conclusion

Lessons for AI4SE. First, generated SQL should be treated as a software artifact requiring traceability, regression tests, and structural checks. Second, preserving intermediate artifacts makes it possible to investigate the first wrong commitment rather than inspecting only the final SQL. During the one-month practice operation, this distinction helped determine whether a problem originated in column grounding, table linking, join planning, or filtering and exposed the concrete item requiring attention. Third, operational acceptance requires more than schema correctness: the practice operation revealed inefficient SQL with equivalent results and omitted implicit predicates, including rules for excluding logically deleted records. These observations motivate converting DA/DBA-reviewed SQL into reusable reference artifacts and regression cases so that operational knowledge can accumulate beyond the initial 69-query suite.

Operational implication. The proposed workflow does not treat human review as a temporary exception or claim immediate full automation. Previously unseen or high-risk requests remain subject to DA/DBA review, while approved or corrected SQL expands the reference corpus for subsequent diagnosis and regression testing. As validated coverage grows, automation can be broadened within repeatedly tested regions while retaining review for novel or operationally sensitive cases. This provides a governed path toward greater automation without equating executability or result equivalence with production readiness.

Threats to validity. The quantitative study covers one financial domain and 69 queries, while the one-month practice operation provides qualitative operational observations rather than a controlled accuracy estimate. Results depend on metadata quality and the catalog/join constraints. Strict table-set matching may penalise valid alternative structures or necessary bridge tables, and CLA/TLA/SER do not establish denotation equivalence or query-plan efficiency. Gold Reverser errors may also propagate into reference artifacts, while offline replay may require minimal adaptation of production SQL constructs. The reported evidence therefore supports failure localization and operational correctability, not unrestricted autonomous execution.

Conclusion. I contribute artifact-level failure localization that compares inspectable commitments with reviewable references derived from administrator-approved SQL. Paired replay of 69 queries shows improved schema commitments and fewer structural failures, while a subsequent one-month practice operation illustrates how the same artifacts can expose the stage and concrete decision responsible for a problem in previously unseen requests. The operation also reveals that result equivalence and schema linking alone do not ensure production readiness because SQL may remain inefficient or omit implicit business rules. I therefore propose continuously incorporating DA/DBA-reviewed SQL into reference artifacts and regression assets as a governed path from human validation toward progressively broader automation.

References

1. Ma, W., et al.: Rethinking testing for llm applications: Characteristics, challenges, and a lightweight interaction protocol. arXiv preprint arXiv:2508.20737 (2025). https://doi.org/10.48550/arXiv.2508.20737
2. Wang, B., Shin, R., Liu, X., Polozov, O., Richardson, M.: Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL). pp. 7567–7578 (2020). https://doi.org/10.18653/v1/2020.acl-main.677
3. Wang, C., et al.: Robust text-to-sql generation with execution-guided decoding. arXiv preprint arXiv:1807.03100 (2018). https://doi.org/10.48550/arXiv.1807.03100
4. Wang, Y., Liu, P., Yang, X.: Linkalign: Scalable schema linking for real-world large-scale multi-database text-to-sql. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 977–991 (2025). https://doi.org/10.18653/v1/2025.emnlp-main.51
5. Yu, T., et al.: Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 3911–3921 (2018). https://doi.org/10.18653/v1/D18-1425
6. Zeller, A., Hildebrandt, R.: Simplifying and isolating failure-inducing input. IEEE Transactions on Software Engineering **28**(2), 183–200 (2002). https://doi.org/10.1109/32.988498
7. Zhou, J., et al.: Guardian: Safeguarding llm multi-agent collaborations with temporal graph modeling. arXiv preprint arXiv:2505.19234 (2025). https://doi.org/10.48550/arXiv.2505.19234